# AI ASSISTED CODING

# ASSIGNMENT – 8.1

**NAME:** Varshitha. G

**HTNO:** 2303A51103

**Task Description #1 (Password Strength Validator – Apply AI in Security Context)**

• Task: Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.

• Requirements:

o Password must have at least 8 characters.

o Must include uppercase, lowercase, digit, and special character.

o Must not contain spaces.

Example Assert Test Cases:

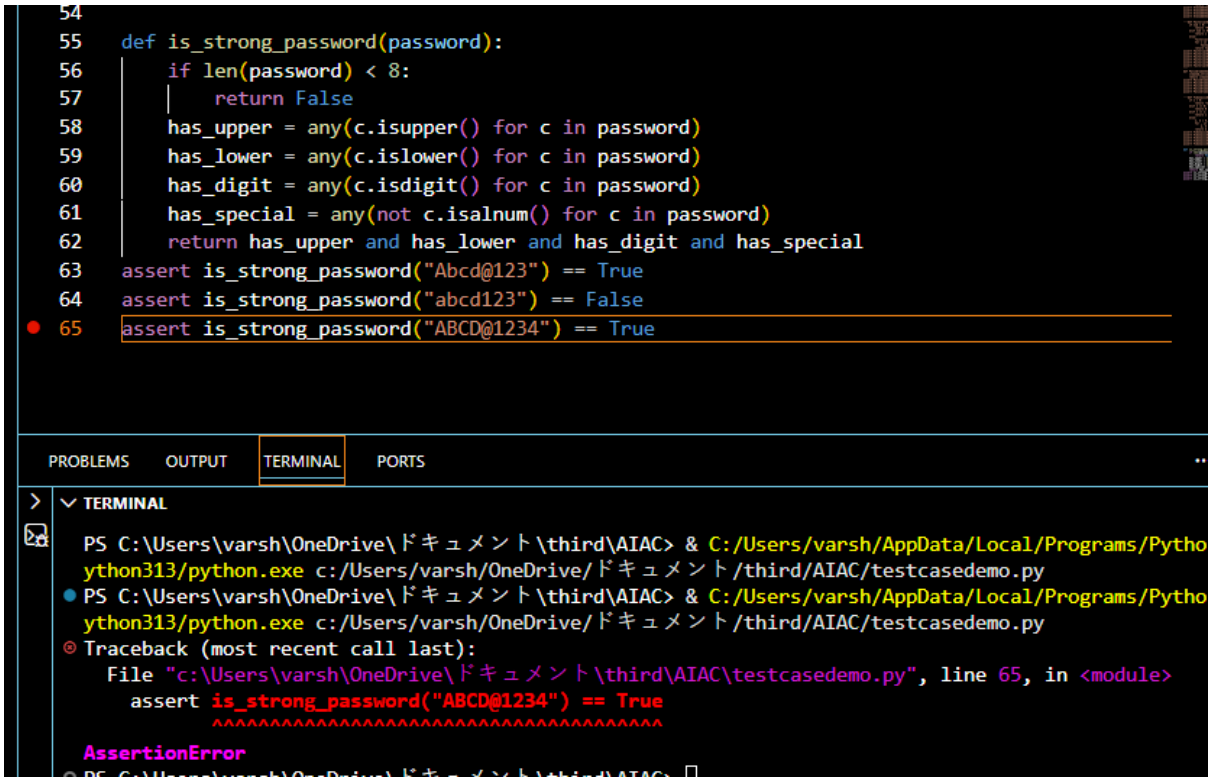
assert is_strong_password("Abcd@123") == True

assert is_strong_password("abcd123") == False

assert is_strong_password("ABCD@1234") == True

Expected Output #1:

• Password validation logic passing all AI-generated test cases.



**Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)**

• Task: Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.

• Requirements:

o Classify numbers as Positive, Negative, or Zero.

o Handle invalid inputs like strings and None.

o Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

assert classify_number(10) == "Positive"

assert classify_number(-5) == "Negative"

assert classify_number(0) == "Zero"

Expected Output #2:

• Classification logic passing all assert tests.

```
67  def classify_number(n):
68      if n > 0:
69          return "Positive"
70      elif n < 0:
71          return "Negative"
72      else:
73          return "Zero"
74  assert classify_number(10) == "Positive"
75  assert classify_number(-5) == "Negative"
76  assert classify_number(0) == "Zero"
```

```
PROBLEMS  OUTPUT  TERMINAL  PORTS
TERMINAL
  PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh
  ython313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/
  PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

**Task Description #3 (Anagram Checker – Apply AI for String Analysis)**

• Task: Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.

• Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

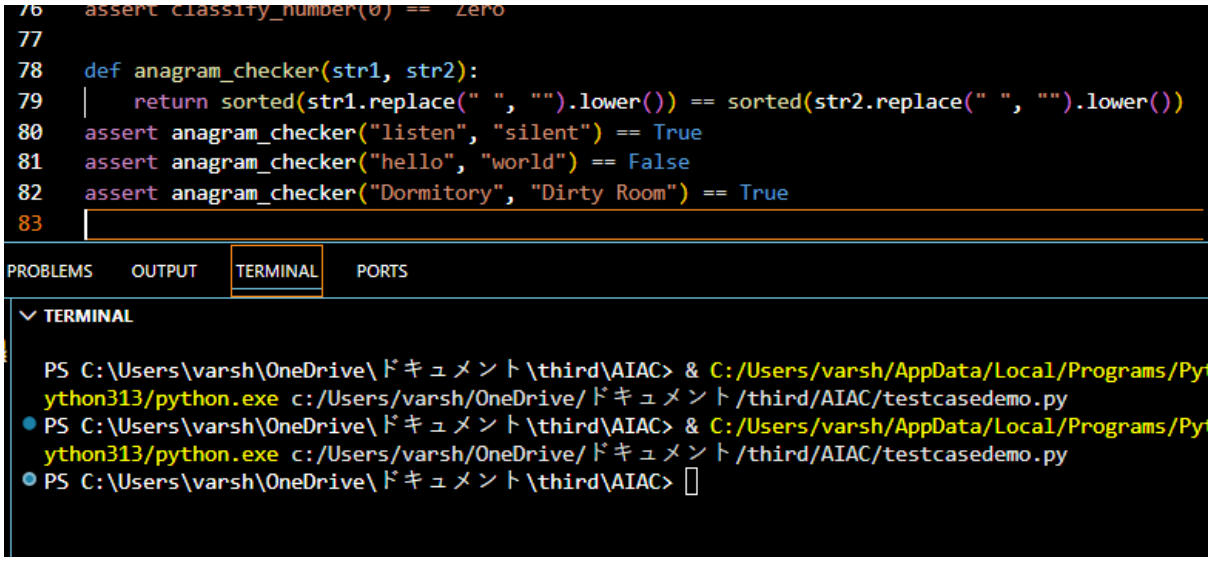
assert is_anagram("listen", "silent") == True

assert is_anagram("hello", "world") == False

assert is_anagram("Dormitory", "Dirty Room") == True

Expected Output #3:

• Function correctly identifying anagrams and passing all AI- generated tests.




**Task Description #4 (Inventory Class – Apply AI to Simulate Real- World Inventory System)**

• Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

• Methods:

o add_item(name, quantity)

o remove_item(name, quantity)

o get_stock(name)

Example Assert Test Cases:

inv = Inventory()

inv.add_item("Pen", 10)

assert inv.get_stock("Pen") == 10

inv.remove_item("Pen", 5)

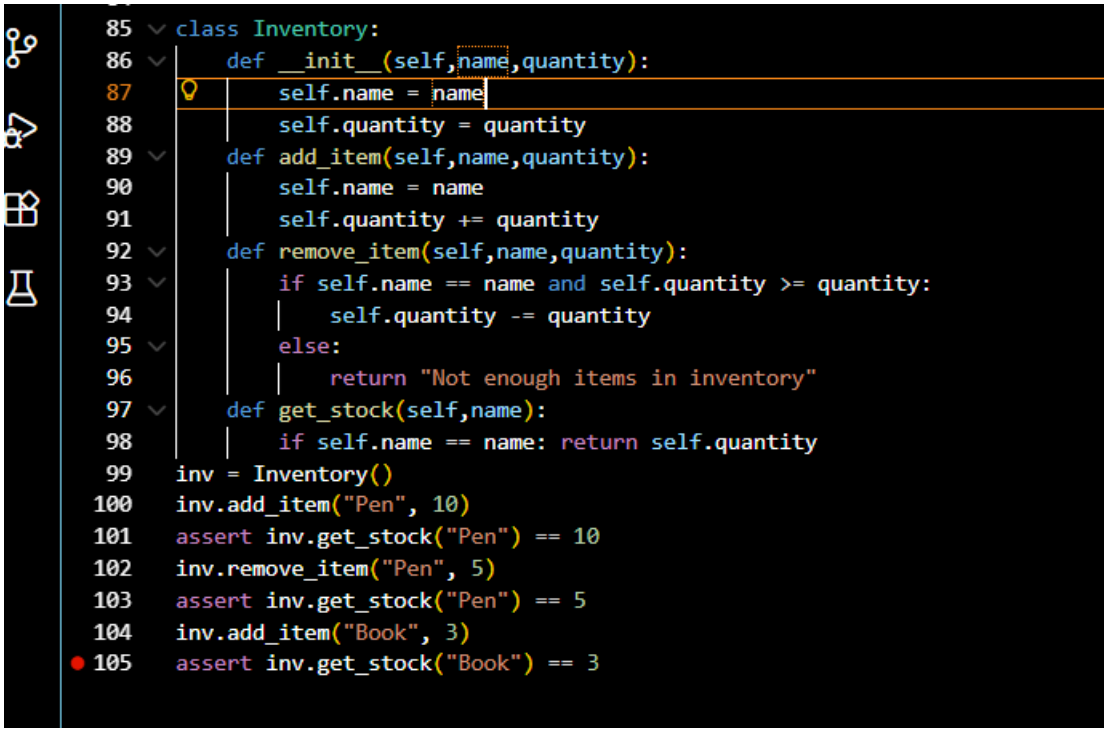
assert inv.get_stock("Pen") == 5

inv.add_item("Book", 3)

assert inv.get_stock("Book") == 3

Expected Output #4:

• Fully functional class passing all assertions.



```
85  class Inventory:
86      def __init__(self,name,quantity):
87          self.name = name
88          self.quantity = quantity
89      def add_item(self,name,quantity):
90          self.name = name
91          self.quantity += quantity
92      def remove_item(self,name,quantity):
93          if self.name == name and self.quantity >= quantity:
94              self.quantity -= quantity
95          else:
96              return "Not enough items in inventory"
97      def get_stock(self,name):
98          if self.name == name: return self.quantity
99  inv = Inventory()
100 inv.add_item("Pen", 10)
101 assert inv.get_stock("Pen") == 10
102 inv.remove_item("Pen", 5)
103 assert inv.get_stock("Pen") == 5
104 inv.add_item("Book", 3)
105 assert inv.get_stock("Book") == 3
```

**Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)**

• Task: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.

• Requirements:

o Validate "MM/DD/YYYY" format.

o Handle invalid dates.

o Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

assert validate_and_format_date("10/15/2023") == "2023-10-15"

assert validate_and_format_date("02/30/2023") == "Invalid Date"

assert validate_and_format_date("01/01/2024") == "2024-01-01"

Expected Output #5:

• Function passes all AI-generated assertions and handles edge cases.